博弈论竞赛板子

本板子默认 C++17，所有 64 位整数统一写作 `ll`。代码块内只保留“传什么参数、返回什么”的短注释，完整条件和证明思路放在代码块外，方便赛场直接复制。

0. 先判断模型

模型	关键特征	直接使用
普通 Nim	每次只改一堆，取任意正数	全部异或 `xor`
反常 Nim	取走最后一颗者输	小堆特判，其余看异或
巴什	一堆，每次取 `1..m`	`n%(m+1)`
区间取石子	每次取 `a..b`	`n%(a+b)<a` 为败态
威佐夫	两堆，单堆取或两堆同取	黄金分割 Beatty 数
斐波那契	首次不能取完，此后至多取上次两倍	`n` 为 Fibonacci 数时败
欧几里得博弈	两数间轮流减去较大数的正倍数	商至少为 `2` 或整除时当前胜
Nimble	棋子可沿一条线向左移动	奇数个棋子所在位置异或
Silver Dollar	棋子不可相互越过，只能向左走	交替间隔异或
一般有限无环游戏	状态有向边，不能无限走	SG / `mex`
多个独立游戏并行	一次只能走其中一个	各 SG 异或
反常一般游戏	无法行动者胜	反常 SG，不能只看普通 SG
有环状态图	可能循环或和棋	反向拓扑判 Win/Lose/Draw
带分值零和游戏	不只判断胜负	DAG Negamax / Minimax
多个独立游戏并行	一次只能走其中一个	各 SG 异或
反常一般游戏	无法行动者胜	反常 SG，不能只看普通 SG

先确认“是否必须取、取完后谁赢、一次能改几堆、游戏是否可能无限进行”。规则不同，结论可能完全相反。

还要区分三件事：双方可选操作完全相同才是 `impartial`，否则是 `partisan`，不能直接套 SG；“无法行动者输”是 normal play，“无法行动者赢”是 misere play；状态有环时还要明确无限循环是胜、负还是和棋。

1. 公共代码与 mex

```
#include <bits/stdc++.h>
```

```cpp
using namespace std;
using ll = long long;
using ull = unsigned long long;
using i128 = __int128_t;

const ll INF = (1LL << 62);

// 返回非负整数集合的 mex；a 可含重复值和负值
int mex(vector<int> a) {
    int n = (int)a.size();
    vector<int> v(n + 1);
    for (int x : a) if (0 <= x && x <= n) v[x] = 1;
    for (int i = 0; ; ++i) if (!v[i]) return i;
}
```

若出边数量很多，直接建立 `unordered_set` 会有较大常数。下面的时间戳标记写法将一次 `mex` 降为线性，适合 DFS 求 SG。

```cpp
struct Mex {
    vector<int> mk;
    int tim = 0;
    Mex(int n = 0) : mk(n + 1) {}
    // a: 待取 mex 的值；返回 mex(a)
    int get(const vector<int> &a) {
        ++tim;
        if (tim == INT_MAX) fill(mk.begin(), mk.end(), 0), tim = 1;
        for (int x : a) {
            if (x < 0) continue;
            if (x >= (int)mk.size()) mk.resize(x + 1);
            mk[x] = tim;
        }
        int x = 0;
        while (x < (int)mk.size() && mk[x] == tim) ++x;
        return x;
    }
};
```

SG 值只需要关心 `0..出度`，因此可以把更大的后继值忽略；若已知最大 SG 为 `K`，把标记数组开到 `K+1` 即可进一步省内存。

2. Nim 家族

2.1 普通 Nim

有 `n` 堆，每次任选一堆取走正数，取完最后一颗者胜。设 `s = a[1] xor ... xor a[n]`：`s=0` 时先手必败，否则先手必胜。证明是 Bouton's theorem：从异或非零局面总能走到异或零局面，而异或零局面走一步必变非零。

```cpp
// a: 各堆石子数；返回 true 表示先手必胜
bool nim_win(const vector<ll> &a) {
```

```
    ll s = 0;
    for (ll x : a) s ^= x;
    return s != 0;
}

// a: 各堆石子数；返回 {堆下标, 取走数量}，无解返回 {-1,0}
pair<int, ll> nim_move(const vector<ll> &a) {
    ll s = 0;
    for (ll x : a) s ^= x;
    if (!s) return {-1, 0};
    for (int i = 0; i < (int)a.size(); ++i) {
        ll b = a[i] ^ s;
        if (b < a[i]) return {i, a[i] - b};
    }
    return {-1, 0};
}
```

`nim_move` 返回的下标从 `0` 开始；若题目堆编号从 `1` 开始，输出时加一。为了防止有符号整数移位问题，这里只使用异或和比较，不手动枚举最高位。

2.2 反常 Nim（Misere Nim）

仍是一次只动一堆，但拿走最后一颗者输。若所有堆大小都不超过 `1`，则"先手输"当且仅当非空堆数为奇数；若存在堆大于 `1`，则结论与普通 Nim 相同：`xor=0` 先手负，`xor!=0` 先手胜。反常规则只改变了全是 `0/1` 堆的终局。

```
// a: 各堆石子数；返回 true 表示先手必胜（反常规则）
bool misere_nim_win(const vector<ll> &a) {
    ll s = 0;
    int big = 0, one = 0;
    for (ll x : a) s ^= x, big |= (x > 1), one += (x == 1);
    if (!big) return (one & 1) == 0;
    return s != 0;
}
```

2.3 Moore's Nim-K（一次最多改 `k` 堆）

每次至多选择 `k` 堆，分别取任意正数。对每个二进制位统计所有堆该位的 `1` 的数量 `c[j]`。当且仅当所有 `c[j] % (k+1) == 0` 时是败态。注意这不是普通 Nim 的单个异或；题目若要求"恰好改 `k` 堆"，结论不能直接套用。

```cpp
// a: 各堆石子数；k: 一次最多改几堆；返回 true 表示先手必胜
bool moore_win(const vector<ll> &a, int k) {
    if (k <= 0) return false;
    int c[64] = {};
    for (ll x : a)
        for (int j = 0; j < 63; ++j) c[j] += (x >> j) & 1LL;
    for (int j = 0; j < 63; ++j)
        if (c[j] % (k + 1)) return true;
    return false;
}
```

2.4 阶梯 Nim

第 `i` 级石子只能移到 `i-1` 级，地面上的石子不能再动。标准规则下只需异或奇数级（按题面从 `1` 开始编号）的石子数：异或为零先手败，否则先手胜。

```cpp
// a[1..n]: 第 i 级石子数；返回 true 表示先手必胜
bool stair_win(const vector<ll> &a) {
    ll s = 0;
    for (int i = 1; i < (int)a.size(); ++i) if (i & 1) s ^= a[i];
    return s != 0;
}
```

2.5 Nimble

一条线上有若干棋子，每次任选一枚向左移动任意正距离，不能越过位置 `0`；到达 `0` 后不能再动。若同一位置有多枚棋子，它们是相同子游戏的重复和：只有奇数枚会在异或中留下贡献。因此 `xor(position)`（只统计该位置棋子数为奇数的位置）非零时先手胜。

```cpp
// c[i]: 位置 i 的棋子数；返回 true 表示先手必胜
bool nimble_win(const vector<ll> &c) {
    ll s = 0;
    for (ll i = 0; i < (ll)c.size(); ++i)
        if (c[i] & 1) s ^= i;
    return s != 0;
}
```

若输入直接给出每枚棋子的位置数组 `p`，则把所有 `p[i]` 异或即可；同一位置出现偶数次会抵消。

2.6 Silver Dollar Game

有若干枚棋子排在一条线上，棋子不能相互越过；每次任选一枚向左移动任意正距离，不能移到另一枚棋子上。将位置升序记为 `p[0..n-1]`，只对偶数下标棋子统计它前面的空格数：`g[0]=p[0]`，`g[i]=p[i]-p[i-1]-1`（`i` 为偶数）。这些间隔组成 Nim，异或为零时先手败。

```cpp
// p: 已排序且互异的非负位置；返回 true 表示先手必胜
bool silver_win(const vector<ll> &p) {
    ll s = 0;
    for (int i = 0; i < (int)p.size(); i += 2) {
        ll l = (i ? p[i - 1] : -1);
        s ^= p[i] - l - 1;
    }
    return s != 0;
}
```

若输入未排序，先 `sort(p.begin(),p.end())`；题面允许棋子重合或允许跳过棋子时，模型已改变，不能使用此公式。

3. 单堆取石子

3.1 巴什博弈

每次取 `1..m`，必须取到最后一颗。`n%(m+1)==0` 为败态。需要输出具体策略时，先手取 `n%(m+1)` 颗即可，之后对手取 `x` 就取 `m+1-x`。

```cpp
// n: 石子数；m: 每次最多取数；返回 true 表示先手必胜
bool bash_win(ll n, ll m) {
    if (n < 0 || m <= 0) return false;
    if (m == LLONG_MAX) return n != 0;
    return n % (m + 1) != 0;
}

// 返回先手第一步应取数；败态返回 0
ll bash_move(ll n, ll m) {
    if (n < 0 || m <= 0) return 0;
    if (m == LLONG_MAX) return n;
    return n % (m + 1);
}
```

3.2 区间巴什

每次必须取 `a..b`，且剩余不足 `a` 时不能行动；拿完最后一颗者胜。令 `r=n%(a+b)`，当 `0<=r<a` 时为败态，否则为胜态。`a=1` 时退化为普通巴什。

```cpp
// n: 石子数；a,b: 每次可取区间 [a,b]；返回 true 表示先手必胜
bool range_bash_win(ll n, ll a, ll b) {
    if (n < 0 || a <= 0 || a > b) return false;
    i128 r = (i128)n % ((i128)a + b);
    return r >= a;
}
```

3.3 减法游戏的周期性

一堆石子允许取集合 `mv` 中的数，取完者胜。若 `mv` 固定且最大值为 `mx`，胜负序列只依赖最近 `mx` 个位置，必然最终周期。小范围直接 DP；需要在线多次询问时先检测周期。

```
// mv: 可取正数集合；n: 询问石子数；返回 true 表示先手必胜
bool sub_win(const vector<int> &mv, int n) {
    if (mv.empty() || n <= 0) return false;
    int mx = *max_element(mv.begin(), mv.end());
    vector<char> w(n + 1);
    for (int i = 1; i <= n; ++i)
        for (int x : mv) if (x <= i && !w[i - x]) { w[i] = 1; break; }
    return w[n];
}
```

若 `n` 极大，记录状态向量 `(w[i-mx+1],...,w[i])` 的首次出现位置即可定位周期；状态数理论上最多 `2^mx`，`mx` 较大时应优先寻找数学规律或使用 SG 的特殊结论。

下面给出 `mx<=63` 的严格状态判重版。它不是猜周期：最近 `mx` 个胜负状态完全决定下一项，因此同一窗口再次出现后，序列必然循环。适合 `n` 很大但实际周期较短的题；最坏状态数仍为 `2^mx`。

```
// mv: 可取正数集合，max(mv)<=63；n: 超大询问
bool sub_win_big(vector<int> mv, ll n) {
    mv.erase(remove_if(mv.begin(), mv.end(), [](int x) { return x <= 0; }),
mv.end());
    sort(mv.begin(), mv.end());
    mv.erase(unique(mv.begin(), mv.end()), mv.end());
    if (mv.empty() || n <= 0) return false;
    int mx = mv.back();
    assert(mx <= 63);
    ull mask = (1ULL << mx) - 1, st = 0;
    unordered_map<ull, int> pos;
    vector<char> w(1, 0);
    for (int i = 0; ; ++i) {
        if ((ll)i == n) return w[i];
        if (i >= mx) {
            auto it = pos.find(st);
            if (it != pos.end()) {
                ll p = it->second, per = i - p;
                return w[p + (n - p) % per];
            }
            pos[st] = i;
        }
        char z = 0;
        for (int d : mv)
            if (d <= i + 1 && !w[i + 1 - d]) { z = 1; break; }
        w.push_back(z);
        st = ((st << 1) | z) & mask;
    }
}
```

3.4 斐波那契博弈

一堆有 `n` 颗，先手第一次不能取完，此后每次最多取上一次的两倍。使用 Zeckendorf 定理可得：当且仅当 `n` 本身是 Fibonacci 数（从 `1,2` 开始）时先手必败。

```
// n: 石子数；返回 true 表示先手必胜
bool fib_win(ll n) {
    if (n <= 0) return false;
    vector<ll> f = {1, 2};
    while (f.back() < n) {
        __int128 z = (__int128)f[f.size() - 1] + f[f.size() - 2];
        if (z > LLONG_MAX) break;
        f.push_back((ll)z);
    }
    return !binary_search(f.begin(), f.end(), n);
}
```

若 `n` 接近 `ll` 上界，生成 Fibonacci 时需用 `__int128` 判断下一项是否溢出；通常题目范围远小于 `9e18`。

```
// n: 石子数；返回先手第一步应取数；败态返回 0
ll fib_move(ll n) {
    if (n <= 1) return 0;
    vector<ll> f = {1, 2};
    while (f.back() <= n) {
        __int128 z = (__int128)f[f.size() - 1] + f[f.size() - 2];
        if (z > LLONG_MAX) break;
        f.push_back((ll)z);
    }
    ll r = n, ans = 0;
    for (int i = (int)f.size() - 1; i >= 0; --i)
        if (f[i] <= r) ans = f[i], r -= f[i];
    // ans==n 恰好说明 n 是 Fibonacci 败态
    return ans == n ? 0 : ans;
}
```

`fib_move` 使用 Zeckendorf 分解，返回分解中最小的 Fibonacci 数；这一步是经典 Fibonacci Nim 的必胜首步。原局面若为 Fibonacci 数，返回 `0` 表示没有合法必胜首步。

3.5 欧几里得博弈

有两个正整数 `a,b`，每次从较大数中减去较小数的正整数倍，结果不能为负；先得到 `0` 的玩家获胜。设 `a>=b>0`：若 `a/b>=2`，当前玩家可以选择合适倍数直接取得主动权并获胜；若 `a%b==0`，当前玩家可直接减成 `0`；否则只能走到 `(b,a%b)`，胜负翻转。递归深度为 `O(log(max(a,b)))`。

```cpp
// a,b: 当前两数；返回 true 表示当前玩家必胜
bool euclid_win(ll a, ll b) {
    if (a <= 0 || b <= 0) return false;
    if (a < b) swap(a, b);
    if (a / b >= 2 || a % b == 0) return true;
    return !euclid_win(b, a % b);
}
```

题面若要求“每次只能减去一次较小数”，这就不是欧几里得博弈，而是普通的减法游戏；必须先核对倍数是否任意。

4. 两堆经典博弈

4.1 威佐夫博弈

每次可以从一堆取任意正数，或同时从两堆取相同正数。设 `x<=y`、`d=y-x`，当 `floor(phi*d)==x` 时为败态，其中 `phi=(1+sqrt(5))/2`。浮点数在 Fibonacci 附近会非常接近整数边界，严格判定不应依赖 `double`。

```cpp
// d: 0<=d<=1e18；返回 floor(phi*d)，全整数精确判定
ll phi_floor(ll d) {
    ll l = d, r = 2 * d;
    i128 z = (i128)5 * d * d;
    while (l < r) {
        ll m = l + (r - l + 1) / 2;
        i128 t = (i128)2 * m - d;
        if (t * t <= z) l = m;
        else r = m - 1;
    }
    return l;
}

// x,y: 0..1e18 两堆石子数；返回 true 表示先手必胜
bool wythoff_win(ll x, ll y) {
    if (x > y) swap(x, y);
    return x != phi_floor(y - x);
}
```

`phi_floor` 二分最大的 `k` 使 `k<=phi*d`，将不等式平方后用 `i128` 比较，复杂度 `O(log d)`，没有浮点误差。败态按 `d=0,1,2,...` 为 `(floor(phi*d), floor(phi*d)+d)`。若输入超过 `1e18`，需要先检查 `2*d` 和 `5*d*d` 的范围，或改用 `boost::multiprecision::cpp_int`。

4.2 两堆异或型扩展

若操作是“任选一堆取石子”，就是普通 Nim；若允许“同时取相同数”，才是威佐夫。不要把“两堆”误认为只需比较差值，先逐条核对操作集合。

5. SG 定理与有向无环游戏

5.1 定义和适用条件

对正常（无法行动者输）、有限、无环的 impartial game：

- `P`（必败）态的所有后继都是 `N`（必胜）态；`N` 态至少有一个后继为 `P` 态；
- 终止局面 `sg=0`；
- `sg(x)=mex({sg(y): x -> y})`；
- 单局 `sg=0` 为败态；
- 多个独立子游戏一次只能走一个，总 SG 是所有子游戏 SG 的异或。

有环时不能直接递归 SG；需要题目保证每步势能下降，或先缩 SCC 后另行分析。

5.2 DAG 上 SG：记忆化 DFS（递归版）

适合图深度不大、代码短的场景。`g[x]` 是从 `x` 可走到的点，点编号 `0..n-1`。

```
struct SG {
    vector<vector<int>> g;
    vector<int> f;
    Mex ms;
    SG(int n = 0) : g(n), f(n, -1), ms(n + 1) {}
    void add(int u, int v) { g[u].push_back(v); } // 有向边 u->v
    int dfs(int x) { // x: 局面；返回 SG(x)
        if (f[x] != -1) return f[x];
        vector<int> a;
        a.reserve(g[x].size());
        for (int y : g[x]) a.push_back(dfs(y));
        return f[x] = ms.get(a);
    }
};
```

5.3 DAG 上 SG：拓扑序（不爆栈）

图深度可能达到 `2e5` 时优先用此版。必须保证输入是 DAG；若拓扑点数小于 `n`，说明有环，不能套用。

```
// g: DAG，点 0..n-1；返回每点 SG，非 DAG 时返回空数组
vector<int> dag_sg(const vector<vector<int>> &g) {
    int n = (int)g.size();
    vector<int> in(n), ord, f(n);
    for (int u = 0; u < n; ++u) for (int v : g[u]) ++in[v];
    queue<int> q;
    for (int i = 0; i < n; ++i) if (!in[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); ord.push_back(u);
        for (int v : g[u]) if (!--in[v]) q.push(v);
    }
    if ((int)ord.size() != n) return {};
    Mex ms(n + 1);
```

```
    for (int k = n - 1; k >= 0; --k) {
        int u = ord[k];
        vector<int> a;
        a.reserve(g[u].size());
        for (int v : g[u]) a.push_back(f[v]);
        f[u] = ms.get(a);
    }
return f;
}
```

5.4 有环状态图：Win / Lose / Draw

普通 SG 要求无环；若状态图可能循环，且题目把无限循环视为和棋，可用反向拓扑传播：终点是 `Lose`；能走到 `Lose` 的点是 `Win`；只有当所有后继都确定为 `Win` 时才是 `Lose`；剩余未确定点为 `Draw`。复杂度 `O(n+m)`，重边也能正确处理。

```
// 返回 0=Lose，1=Win，2=Draw；默认无限循环为和棋
vector<int> game_outcome(const vector<vector<int>> &g) {
    int n = (int)g.size();
    vector<vector<int>> rg(n);
    vector<int> rem(n), st(n, -1);
    queue<int> q;
    for (int u = 0; u < n; ++u) {
        rem[u] = (int)g[u].size();
        for (int v : g[u]) rg[v].push_back(u);
        if (!rem[u]) st[u] = 0, q.push(u);
    }
    while (!q.empty()) {
        int v = q.front(); q.pop();
        for (int u : rg[v]) if (st[u] == -1) {
            if (st[v] == 0) st[u] = 1, q.push(u);
            else if (--rem[u] == 0) st[u] = 0, q.push(u);
        }
    }
    for (int &x : st) if (x == -1) x = 2;
    return st;
}
```

若题目规定“无限循环者胜”或“无限循环者负”，不能直接把 `Draw` 改名，需要按该胜负规则重新建模；上面的传播只对应标准的和棋约定。

5.5 一维状态的 SG 模板

例如“从 `x` 可减去 `mv` 中任意数”。状态上限 `n` 已知时，迭代 DP 比哈希递归更快、更稳定。

```cpp
// n: 最大状态；mv: 可转移的正整数；返回 sg[0..n]
vector<int> sub_sg(int n, const vector<int> &mv) {
    vector<int> f(n + 1);
    Mex ms((int)mv.size() + 2);
    for (int x = 1; x <= n; ++x) {
        vector<int> a;
        for (int d : mv) if (d <= x) a.push_back(f[x - d]);
        f[x] = ms.get(a);
    }
    return f;
}
```

复杂度 `O(n*|mv|)`。若只问胜负，可把 `f` 改成 `char` 并用 `w[x]=any(!w[x-d])`；若需要多个独立棋盘的合并，必须保留完整 SG 值。

6. 常见组合游戏

6.1 Lasker's Nim（Multi-SG）

每次可从一堆取正数，或把一堆 `x>2` 分成两堆非空。单堆 SG 为：

`sg(x)=x-1 (x%4=0)`，`x (x%4=1,2)`，`x+1 (x%4=3)`。

空堆 `x=0` 的 SG 定义为 `0`，上面的分段式默认 `x>0`。

多个堆的答案仍为 SG 异或。注意"分裂后两个子堆独立"是使用 Multi-SG 的前提。

```cpp
// x: 单堆大小；返回该堆 SG
ll lasker_sg(ll x) {
    if (x == 0) return 0;
    switch (x & 3) {
        case 0: return x - 1;
        case 3: return x + 1;
        default: return x;
    }
}

// a: 各堆大小；返回 true 表示先手必胜
bool lasker_win(const vector<ll> &a) {
    ll s = 0;
    for (ll x : a) s ^= lasker_sg(x);
    return s != 0;
}
```

若 `x` 可能超过 `int`，SG 值也可能超过 `int`；此时把返回类型改成 `ll`，但题面常见范围足以使用 `int`。

6.2 通用分裂游戏（Multi-SG）

若一个状态 `x` 的一步操作会把它分成两个互不影响的状态 `(a,b)`，该后继的 SG 不是单个值，而是 `sg[a] xor sg[b]`；分成更多部分时继续全部异或，最后对所有合法操作的结果取 `mex`。矩形切割、删点后分成左右两段、拆堆游戏都常用此式。

下面要求所有后继 `a,b<x`，因此可按状态编号递推；`sp[x]` 列出状态 `x` 一步可分成的所有二元组。只产生一个子游戏的操作可写成 `(a,0)`，并约定 `sg[0]=0`。

```
// sp[x]: x 可一步分成的 (a,b)；返回 sg[0..n-1]
vector<int> split_sg(const vector<vector<pair<int, int>>> &sp) {
    int n = (int)sp.size();
    vector<int> f(n);
    Mex ms(n + 1);
    for (int x = 1; x < n; ++x) {
        vector<int> a;
        a.reserve(sp[x].size());
        for (auto [u, v] : sp[x]) a.push_back(f[u] ^ f[v]);
        f[x] = ms.get(a);
    }
    return f;
}
```

6.3 树上删边（Green Hackenbush）

有根树中每次删一条边，根以下被断开的整棵子树消失，删掉最后一条边者胜。令叶子 SG 为 `0`，非叶子 `sg(u)=xor(sg(v)+1)`。根 SG 非零则先手胜。这里的“+1”是边贡献，不是节点编号。

有些题解把状态写成 `h(u)=sg(u)+1`，于是叶子 `h=1`，并写成“根 `h!=1` 先手胜”；这与本节的“标准根 SG `!=0` 先手胜”完全等价。不要把两种记号的 `1` 和 `0` 混用。

```
// g: 无向树；root: 根；返回根 SG，迭代避免深树爆栈
int tree_cut_sg(const vector<vector<int>> &g, int root = 0) {
    int n = (int)g.size();
    vector<int> p(n, -1), ord = {root};
    for (int i = 0; i < (int)ord.size(); ++i) {
        int u = ord[i];
        for (int v : g[u]) if (v != p[u]) p[v] = u, ord.push_back(v);
    }
    vector<int> f(n);
    for (int i = n - 1; i >= 0; --i) {
        int u = ord[i];
        for (int v : g[u]) if (p[v] == u) f[u] ^= f[v] + 1;
    }
    return f[root];
}
```

若一条边代表长度为 `w` 的连续竹节（或等价的 `w` 条串联边），把递推中的 `+1` 换成 `+w`；普通树就是 `w=1`。

```cpp
struct WEdge { int v; ll w; };

// g: 无向树，边权 w>=1 表示串联边数；返回根 SG
ll tree_cut_sg_w(const vector<vector<WEdge>> &g, int root = 0) {
    int n = (int)g.size();
    vector<int> p(n, -1), ord = {root};
    for (int i = 0; i < (int)ord.size(); ++i) {
        int u = ord[i];
        for (auto e : g[u]) if (e.v != p[u]) p[e.v] = u, ord.push_back(e.v);
    }
    vector<ll> f(n);
    for (int i = n - 1; i >= 0; --i) {
        int u = ord[i];
        for (auto e : g[u]) if (p[e.v] == u) f[u] ^= f[e.v] + e.w;
    }
    return f[root];
}
```

6.4 无向图删边与 Fusion Principle

图连通、指定根，每次删边，根不可断开部分继续游戏。对每个奇环缩成“一个点加一条边”，偶环缩成“一个点”，环上的所有外接边接到新点；重复缩环后得到树，再使用上一节的树上删边 SG。实现缩环通常需要边双连通分量/并查集，题面若图含复杂重边，必须先确认是否允许删桥、是否保留根侧，不能直接照抄树公式。

6.5 反常 SG（Misere 组合游戏）

普通 SG 只适用于“无法行动者输”。反常规则下不存在对任意游戏都成立的“异或 SG”黑箱。只有满足特定结构（如反常 Nim、每个子游戏 SG 仅 `0/1` 的若干定理）才能使用简化结论。遇到一般反常 DAG，应做状态 DP：`win(x)=` 是否存在一步到达对手必败态，终态按题意初始化为胜态。

```cpp
// g: DAG；lose_end=true 表示“无法行动者胜”（反常规则）
// 返回 win[0..n-1]，true 表示当前玩家胜
vector<char> misere_dag(const vector<vector<int>> &g, bool lose_end = true) {
    int n = (int)g.size();
    vector<int> in(n), ord;
    for (int u = 0; u < n; ++u) for (int v : g[u]) ++in[v];
    queue<int> q;
    for (int i = 0; i < n; ++i) if (!in[i]) q.push(i);
    while (!q.empty()) {
        int u = q.front(); q.pop(); ord.push_back(u);
        for (int v : g[u]) if (!--in[v]) q.push(v);
    }
    if ((int)ord.size() != n) return {};
    vector<char> w(n);
    for (int k = n - 1; k >= 0; --k) {
        int u = ord[k];
        if (g[u].empty()) w[u] = lose_end;
        else {
            w[u] = 0;
            for (int v : g[u]) if (!w[v]) { w[u] = 1; break; }
```

```
        }
    }
    return w;
}
```

上面参数名 `lose_end` 表示"无路可走者是否获胜"，设为 `true` 即反常规则；普通规则的终态胜负应直接使用 `w[u]=0`，不要误以为把参数改成 `false` 就能覆盖所有带终止奖励的游戏。

Anti-SG 定理（反常组合游戏的特例）

对若干个互不影响的正常 impartial 子游戏，改为"无法行动者胜"时，若每个子游戏的普通 SG 值已知，则有以下经典结论：

- 所有 SG 值均不超过 `1`：总 SG 异或为 `0` 时先手胜，否则先手负；
- 至少一个 SG 值大于 `1`：总 SG 异或不为 `0` 时先手胜，否则先手负。

这正是反常 Nim 结论的推广。它要求全局操作仍然是"一次只走一个子游戏"；若一次可以同时改变多个子游戏，必须重新做状态 DP。

```
// f: 各子游戏的普通 SG；返回 true 表示反常规则下先手必胜
bool anti_sg_win(const vector<int> &f) {
    int s = 0;
    bool big = false;
    for (int x : f) s ^= x, big |= (x > 1);
    return big ? (s != 0) : (s == 0);
}
```

6.6 Every-SG（同时移动所有可动棋子）

某些题目规定每回合必须把"所有当前仍可移动的棋子"各走一步，无法移动的棋子停在终点；当所有棋子都不能移动时游戏结束。该模型不是普通的"每次选择一个子游戏"，不能异或各棋子的 SG。题面若明确采用 Every-SG 定义，通常令 `step` 为从当前局面到结束所需的最大回合数（按题目给出的可行走法取最优），结论是 `step` 为奇数先手胜、偶数先手负。实现时先按题意求 DAG 上的最长可行步数；若棋子之间有相互影响，需把联合状态建图后再做 DP。

6.7 多个独立局面的合并

只要每回合只能选择一个局面操作，且各局面互不影响，就可以直接异或 SG；这是 Sprague-Grundy 定理的实际接口。

```
// f: 每个独立游戏的 SG；返回 true 表示先手必胜
bool sum_sg_win(const vector<int> &f) {
    int s = 0;
    for (int x : f) s ^= x;
    return s != 0;
}
```

7. 状态压缩与博弈 DP

当状态数较小、操作会改变多个元素或存在重复局面时，直接对状态做记忆化 minimax 最稳妥。`win(s)` 表示当前玩家是否能获胜；终态普通规则为 `false`。

```
// nxt(s): 返回从状态 s 可到达的状态；普通规则，终态为负
unordered_map<int, char> memo;
function<char(int)> win = [&](int s) -> char {
    auto it = memo.find(s);
    if (it != memo.end()) return it->second;
    auto v = nxt(s); // 题目自行实现：vector<int> nxt(int)
    for (int t : v) if (!win(t)) return memo[s] = 1;
    return memo[s] = 0;
};
```

若状态能编码到 `[0,2^k)`，用 `vector<int8_t>` 代替哈希表，常数更小；若存在环，必须先证明每次操作使势能下降，或使用三态 DFS（`0=未访问,1=递归中,2=已确定`）并按题面处理和局/无限循环。

7.1 子集游戏常用编码

```
// mask: 当前集合；返回最低位 1 的位置
int bit_id(int x) { return __builtin_ctz((unsigned)x); }
// mask: 当前集合；枚举所有非空子集 sub
for (int sub = mask; sub; sub = (sub - 1) & mask) {
    // 使用 sub
}
```

枚举所有子集总复杂度 `O(3^k)`（对所有 mask 和其子集总和），单个 mask 的子集枚举为 `O(2^{popcount(mask)})`。

7.2 带分值的零和博弈：Negamax

如果题目不是只问胜负，而是双方最大化自己的最终得分、且一方得分等于另一方的相反数，可定义 `val(s)` 为“当前轮到行动者相对于对手的最优分差”。终止态返回题目给出的分差；非终止态满足 `f(s)=max(-f(t))`。要求状态图无环或每步势能严格下降。

```
struct Negamax {
    // nxt(s): 后继；ed(s): 是否终止；score(s): 终止态当前视角分差
    function<vector<int>(int)> nxt;
    function<bool(int)> ed;
    function<ll(int)> score;
    unordered_map<int, ll> mp;

    // s: 当前状态；返回当前玩家能保证的分差
    ll dfs(int s) {
        auto it = mp.find(s);
        if (it != mp.end()) return it->second;
        if (ed(s)) return mp[s] = score(s);
        ll r = -INF;
        for (int t : nxt(s)) r = max(r, -dfs(t));
        return mp[s] = r;
```

```
    }
};
```

若只需判断胜负，优先使用前面的 `win`；若状态图有环，不能用这个递归直接取最大值，应先明确无限对局的收益规则。

8. 取胜策略如何输出

判定先手胜后，通常需要输出一步合法操作。通用原则是枚举所有后继：找到一个使对手状态为败（普通规则 `sg=0` 或 `win=false`）的后继即可。

- Nim：用 `a[i]^s < a[i]` 找到目标堆，剩余 `a[i]^s`。
- SG：枚举后继 `y`，找 `sg(y)=0`；若是组合游戏，目标是总异或变为 `0`。
- 树上删边：删除使根 SG 变为 `0` 的那条边；可在 DFS 后枚举每条边重算或维护子树 SG。

输出策略时要再次检查“取走数量为正”“不能取空”“是否允许同时操作多堆”等边界，不能只根据胜负公式输出一个未经验证的操作。

9. 复杂度与易错点

1. `xor` 的运算优先级低于比较但高于赋值，复杂表达式建议加括号。
2. 普通 SG 的终态一定是 `0`；反常游戏终态按题意定义，不能机械套 SG。
3. DAG SG 的边方向必须是“当前状态 -> 下一状态”，拓扑逆序计算。
4. 递归 DFS 深度可达 `n` 时改用拓扑序或显式栈。
5. `mex` 至少覆盖 `0..出度`；使用时间戳数组时不要让旧标记与新轮次冲突。
6. Moore's Nim-K 的条件是每个二进制位计数对 `k+1` 取模为零，且操作是“至多 `k` 堆”。
7. 本板子的威佐夫整数判定要求输入不超过 `1e18`；更大数据需用 `cpp_int`，不要退回 `double`。
8. 有环游戏、可能无限游戏、带“和棋/重复局面”规则的游戏，不属于普通 SG 的直接适用范围。
9. “走过的点不能再走”等规则依赖历史，状态必须包含已访问集合；只拿当前顶点做 SG 会把不同局面错误合并。
10. “至多改 `k` 堆”与“恰好改 `k` 堆”不是同一模型，后者不能使用 Moore's Nim-K 结论。
11. `__builtin_ctz(0)` 未定义，调用 `bit_id` 前必须保证状态非零。
12. `P/N`、SG、Minimax 都默认双方采用最优策略；随机策略或概率转移应改用期望 DP / MDP。

10. 速查

普通 Nim	xor(a[i]) == 0 败
反常 Nim	全<=1: 1 的个数为奇数败；否则 xor==0 败
巴什	n%(m+1)==0 败
区间巴什	n%(a+b)<a 败
威佐夫	x<=y 且 floor(phi*(y-x))==x 败
斐波那契	n 为 Fibonacci(1,2,...) 败
树上删边	sg[u]=xor(sg[v]+1)，根 sg!=0 胜
普通 DAG	sg=mex(后继 SG)，总 SG 异或
Moore Nim-K	每位 1 的个数 % (k+1) 全为 0 败